

# Neural differential distinguishers for GIFT-128 and ASCON

Dongsu Shen<sup>a,b,c</sup>, Yijian Song<sup>a,b,c,\*</sup>, Yuan Lu<sup>d</sup>, Saiqin Long<sup>e</sup>, Shujuan Tian<sup>a,b,c</sup>

<sup>a</sup> School of Computer Science & School of Cyberspace Security, Xiangtan University, Xiangtan, 411105, China

<sup>b</sup> Hunan International Scientific and Technological Cooperation Base of Intelligent network, Xiangtan University, Xiangtan, 411105, China

<sup>c</sup> Key Laboratory of Hunan Province for Internet of Things and Information Security, Xiangtan University, Xiangtan, Hunan, 411105, China

<sup>d</sup> Institute of Software, Chinese Academy of Sciences, Beijing, 100190, China

<sup>e</sup> College of Information Science and Technology, Jinan University, Guangzhou, 510632, China

## ARTICLE INFO

### Keywords:

Neural distinguisher  
Differential cryptanalysis  
GIFT-128  
ASCON

## ABSTRACT

In CRYPTO 2019, Gohr first introduced a neural differential distinguisher for round-reduced SPECK32/64 to distinguish ciphertext pairs with a specific input difference from random data, indicating a promising approach to the cryptanalysis of lightweight ciphers. This paper proposes a new neural differential distinguisher model for GIFT and ASCON, both of them are the finalists of the NIST Lightweight Cryptography Competition with ASCON being announced as the winner in February 2023. In our model, we utilize the score distribution of multiple ciphertext differences instead of a single ciphertext pair for classification, which is different from Gohr's model. First, we construct neural distinguishers to evaluate the scores for ciphertext differences. Then we train neural networks to classify the distribution of scores, which is more efficient than the traditional statistical tests such as the Kolmogorov–Smirnov test. Based on the proposed model, we improve the prediction accuracy from 55.42% to 99.36% and from 50.69% to 69.25% for 7-round GIFT-128 and 4-round ASCON-PERMUTATION respectively. Compared with the previous neural distinguisher for ASCON, our distinguisher covers an additional round and attains higher accuracy. The experimental results show that better results can be obtained with the lower hamming weight input difference, and the method presented in this paper can improve the effectiveness of cryptanalysis with neural networks.

## 1. Introduction

The Internet of Things (IoT) is rapidly developing and widely used in various fields. Since IoT devices are typically resource-constrained, such as sensors and RFIDs, the ciphers implemented on these devices must satisfy both security and performance requirements within this environment. This entails efficient design and implementation of ciphers [1]. To fulfill these requirements, lightweight ciphers have been introduced [2–5] as a class of ciphers that have compact implementations for security. Due to their suitability for IoT devices, lightweight ciphers have gained prominence [6].

Compared with traditional ciphers such as the AES [7], lightweight ciphers are characterized by lower power consumption, smaller area requirement, less processing time, and cheaper cost [8]. Lightweight cryptography is a subcategory of cryptography that aims to provide customized solutions for constrained devices [9]. National Institute of Standards and Technology (NIST) [10] announced the first and second-round candidates for the Lightweight Cryptography Competition in April 2019 and August 2019, respectively. In March 2021,

NIST [11] announced 10 finalists for the final round including ASCON and GIFT-COFB. And in February 2023, ASCON was selected as the winner.

The ability of ciphers to ensure security is of utmost importance. Differential cryptanalysis [12] and linear cryptanalysis [13] are commonly used in modern cryptanalysis. Differential cryptanalysis uses automated techniques, e.g. mixed integer linear programming (MILP) [14], for the identification of differential transitions. By identifying the most probable differential transition, analysts can construct a distinguisher to discern real ciphertext pairs from random ones. Subsequently, the obtained information can be leveraged to execute key recovery attacks.

Machine learning has made significant advancements in recent decades and has found applications in various fields, such as image processing [15], natural language processing [16], and autonomous driving [17]. The combination of machine learning and cryptography was first explored in ASIACRYPT'91 [18]. Machine learning was applied to side-channel attacks in [19,20] for the first time, and achieved better results than traditional methods.

\* Corresponding author at: School of Computer Science & School of Cyberspace Security, Xiangtan University, Xiangtan, 411105, China.

E-mail addresses: [dschen@xtu.edu.cn](mailto:dschen@xtu.edu.cn) (D. Shen), [songyijian33@163.com](mailto:songyijian33@163.com) (Y. Song), [luyuan@iscas.ac.cn](mailto:luyuan@iscas.ac.cn) (Y. Lu), [saiqinlong@jnu.edu.cn](mailto:saiqinlong@jnu.edu.cn) (S. Long), [stianwork@xtu.edu.cn](mailto:stianwork@xtu.edu.cn) (S. Tian).

<https://doi.org/10.1016/j.jisa.2024.103758>

Recent research has increasingly considered applying machine learning to differential cryptanalysis, particularly in the context of lightweight ciphers. Gohr [21] proposed an 8-round distinguisher for SPECK32/64 constructed with a deep residual network at CRYPTO 2019, and performed efficient key recovery attacks with it. Subsequently, Benamira et al. [22] analyzed the underlying mechanisms of Gohr's distinguishers in detail from the perspectives of cryptanalysis and machine learning at EUROCRYPT in 2021, they pointed out that the network constructed a highly accurate approximated Difference Distribution Table (DDT), and proposed a method for improving Gohr's distinguishers. In another study, Su et al. [23] constructed a neural differential distinguisher for the 9-round lightweight cipher Simon32/64 based on the polytopic differential attack and performed key recovery attacks on 11 rounds of Simon32/64.

Baksi et al. proposed two training models for constructing machine learning-based differential distinguishers in [24]. They constructed distinguishers for four lightweight ciphers, including one for 3-round ASCON-PERMUTATION using  $2^{19}$  training data, and compared the training effects of different machine learning models. In a related study, Rajan et al. [25] further developed Baksi's machine learning models of differential distinguisher and trained several distinguishers on 6-round GIFT-COFB with Wang's input differences [26].

Chen et al. [27] considered features derived from multiple ciphertext pairs to construct distinguishers and improved the accuracy of distinguishers for SPECK32/64 with their new model. Bao et al. [28] provided explicit rules that can be used alongside DDTs to improve full DDT-based distinguishers, and conducted a 14-round key recovery attack on SPECK32/64 successfully by introducing related-key differences to neural distinguishers.

Liu et al. [29] proposed two models of neural distinguishers utilizing a new input data generation method. With these new models, they constructed distinguishers for the Speck and Simon families, achieving enhanced performance compared to prior approaches. Teng et al. [30] pointed out that the distinguishing probability of neural distinguishers is significantly influenced by the designs of round functions, and presented some variants of that to compare their resistance against neural distinguishers.

**Motivations:** Some previous neural distinguishers take a pair of ciphertexts or the difference between the two ciphertexts as input, and then output a real-valued score representing the probability that the ciphertext pair is real. The limitation of these distinguishers is that the features among ciphertext pairs are not exploited, their distinguishing only depends on the features derived from a single ciphertext pair, which are not robust. Based on this, we will utilize the score distribution of multiple ciphertext differences instead of a single ciphertext difference for classification.

In this paper, we propose a new model to improve the accuracy of neural distinguishers and demonstrate our model on round-reduced GIFT and ASCON. Both GIFT and ASCON are the finalists in the NIST Lightweight Cryptography (LWC) Competition, with ASCON emerging as the winner in the final round. The main contributions are listed as follows:

- A new method is proposed to distinguish ciphertext differences, by considering multiple ciphertext differences as a single input. We first train a neural distinguisher for the round-reduced cipher, which can output a score representing the probability that the ciphertext difference is real. Then we train another neural network to classify the score distributions of real and random differences. Thus, ciphertext differences can be distinguished by identifying the distribution of scores evaluated by the neural distinguisher. This method improves the accuracy of neural distinguishers that distinguish one ciphertext difference each time.

- Neural distinguishers learn the distribution of ciphertext differences instead of following a single differential trail, therefore the differential transition with the highest probability may not necessarily provide the best learning result. We conduct some experiments to compare the differences with hamming weights of 1 to 5, and the results suggest that starting with an input difference with hamming weight 1 is more likely to yield high accuracy.
- We compare the accuracy of the existing distinguishers model with ours on GIFT-128 and ASCON-PERMUTATION. Although the existing model takes multiple ciphertext pairs as input, our new model achieves higher accuracy. Because our distinguishers do not overfit severely like the existing model as the group size increases.

This paper will be structured as follows. Section 2 briefly introduces the preliminaries of differential cryptanalysis, GIFT-128 and ASCON-PERMUTATION. We propose our model of neural distinguishers in Section 3 and provide a comparison with previous neural distinguishers. In Section 4, we present our neural distinguishers for GIFT-128 and ASCON-PERMUTATION, and discuss the influence of input differences with different hamming weights. Then we perform some experiments to compare the existing model with ours. Finally, Section 5 gives a summary of our work.

## 2. Preliminaries

### 2.1. Specification of differential cryptanalysis

Biham and Shamir [12] proposed differential cryptanalysis in 1991, which is a chosen-plaintext attack that studies the characteristics of the differential transition in the encryption process. It aims to recover part of the key bits by analyzing how the difference between plaintext pairs influences the difference between ciphertext pairs.

Now let  $E : \mathbb{F}_{\{0,1\}}^n \rightarrow \mathbb{F}_{\{0,1\}}^n$  be a cryptographic function and  $P_0, P_1$  are two inputs for  $E$  with the input difference  $\delta_a = P_0 \oplus P_1$ . Let  $C_0 = E(P_0), C_1 = E(P_1)$  with the output difference  $\delta_b = C_0 \oplus C_1$ . We refer to  $\delta_a \rightarrow \delta_b$  as the differential transition, and the differential transition probability  $\Pr(\delta_a \rightarrow \delta_b)$  is defined as

$$\Pr(\delta_a \rightarrow \delta_b) = \Pr(E(m) \oplus E(m \oplus \delta_a) = \delta_b) \\ = \frac{|\{m \in \{0,1\}^n : E(m) \oplus E(m \oplus \delta_a) = \delta_b\}|}{2^n}$$

For random permutation on  $\{0,1\}^n$ , if the difference transition  $\delta_a \rightarrow \delta_b$  is given arbitrarily, its average probability  $P = \frac{1}{2^n-1} \approx \frac{1}{2^n}$ .

The Differential Distribution Table (DDT) is constructed by analyzing the differential transition characteristics of the nonlinear component in the encryption, which are usually S-boxes. This is because storing the DDT of an entire cipher with a large state (e.g. 128-bit) is impractical. After that, a  $r$ -round differential distinguisher can be obtained by identifying a high-probability differential characteristic with the chosen input difference. The key recovery attack can then be performed on  $r+1$  round ciphertexts, wherein the last round key is hypothesized and employed to decrypt the ciphertext pairs. The differential distinguisher is used to evaluate the authenticity of the guessed key, and the key with the highest authenticity is selected for further key recovery attacks.

### 2.2. Specification of GIFT-128

GIFT is a lightweight block cipher designed by Banik et al. [31], based on the Substitution-Permutation Network (SPN) structure. It has two versions, namely GIFT-64 and GIFT-128. In this article, we exclusively focus on the 128-bit version, where both the block size and key length are 128 bits. The round function is performed 40 times in GIFT-128.

**Table 1**  
The 4-bit S-box of GIFT-128 as a lookup table.

| $x$    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
|--------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $S(x)$ | 1 | a | 4 | c | 6 | f | 3 | 9 | 2 | d | b | 7 | 5 | 0 | 8 | e |

The round function comprises three steps: SubCells, PermBits, and AddRoundKey. Before performing the first step, The cipher state  $S$  is initialized with the 128-bit plaintext, which is segmented into four 32-bit segments. Likewise, the key state  $KS$  is initialized with the 128-bit key, which is divided into eight 16-bit words.

**SubCells.** This step uses an invertible 4-bit S-box  $S(x)$ . Table 1 shows a lookup table of the S-box represented in hexadecimal notation.

**PermBits.** Each cipher state  $S_i$  applies different 32-bit permutations as shown in Table 2.

**AddRoundKey.** The addition of the round key and round constant is involved in this step. The round key is denoted as  $RK = U \parallel V$ , where  $U, V$  are extracted from the key state with 32 bits. More description about the key schedule can be found at [31].

To add the round key,  $U$  and  $V$  are XORed to the state  $S$  as follows,

$$S_2 \leftarrow S_2 \oplus U,$$

$$S_1 \leftarrow S_1 \oplus V.$$

To add the round constant,  $S_3$  is updated as follows,

$$S_3 \leftarrow S_3 \oplus 0x800000XY,$$

where the byte  $XY = 00c_5c_4c_3c_2c_1c_0$ .

### 2.3. Specification of ASCON-PERMUTATION

ASCON, designed by Dobraunig et al. [32], is a permutation-based lightweight cipher. It is based on the monkeyDuplex [33] construction which is a classical sponge construction. ASCON operates on a state size of 320-bit (consisting of five 64-bit words  $x_0, \dots, x_4$ ), and the main component is a 320-bit permutation instantiated with different constants and the number of rounds. ASCON is the winner of the CAESAR competition and NIST competition.

We only consider the 320-bit permutation for training our neural distinguishers, providing a brief overview of the ASCON-PERMUTATION. For a complete description of the cipher, we refer to [32]. Each round of the permutation consists of three steps: Addition of Constants, Substitution Layer, and Linear Diffusion Layer. These three are iteratively applied for 12 rounds in the ASCON-PERMUTATION  $p^{12}$ .

**Addition of Constants.** In this step  $p_C$ , a round constant  $c_r$  is added to register word  $x_2$  of the state  $S(S = x_0 \parallel x_1 \parallel x_2 \parallel x_3 \parallel x_4)$  in each round. The round constants are stated in Table 3.

**Substitution Layer.** In this layer  $p_S$ , a 5-bit S-box is applied 64 times in parallel to update the state. Each bit of the five 64-bit words  $(x_0, \dots, x_4)$  contributes one bit to each of the 64 S-boxes, where  $x_0$  is the most significant bit. The lookup table of the S-box is shown in Table 4.

**Linear Diffusion Layer.** In this layer  $p_L$ , each 64-bit register word  $x_i$  is diffused by the linear function  $\Sigma_i(x_i)$ ,  $x_i \leftarrow \Sigma_i(x_i)$  for  $i = 0, \dots, 4$ :

$$x_0 \leftarrow \Sigma_0(x_0) = x_0 \oplus (x_0 \ggg 19) \oplus (x_0 \ggg 28)$$

$$x_1 \leftarrow \Sigma_1(x_1) = x_1 \oplus (x_1 \ggg 61) \oplus (x_1 \ggg 39)$$

$$x_2 \leftarrow \Sigma_2(x_2) = x_2 \oplus (x_2 \ggg 1) \oplus (x_2 \ggg 6)$$

$$x_3 \leftarrow \Sigma_3(x_3) = x_3 \oplus (x_3 \ggg 10) \oplus (x_3 \ggg 17)$$

$$x_4 \leftarrow \Sigma_4(x_4) = x_4 \oplus (x_4 \ggg 7) \oplus (x_4 \ggg 41)$$

$x \ggg i$  indicates right-rotation (circular shift) by  $i$  bits of the 64-bit word  $x$ .

### 3. New model of neural distinguishers

Differential distinguisher is a crucial component in the key recovery attack of differential cryptanalysis. The traditional differential distinguisher works with a differential characteristic that provides the highest probability transition from a chosen input difference to a ciphertext difference. Therefore, constructing a traditional differential distinguisher typically involves high data complexity. Neural distinguishers are trained to distinguish ciphertext differences with a fixed input difference from random ciphertext differences. They do not rely on the optimal differential characteristic, which reduces the data complexity of construction.

The proposed new model of neural distinguishers consists of two components: construction of neural distinguishers and prediction with multiple ciphertext differences. Both parts are implemented based on Multi-Layer Perceptron (MLP). The overall architecture of our new model is shown in Fig. 1.

#### 3.1. Construction of neural distinguishers

##### 3.1.1. Data generation

The training and validation data for our neural distinguishers includes the differences between  $r$ -round ciphertext pairs encrypted from plaintext pairs with the chosen difference, as well as  $r$ -round ciphertext differences obtained from random plaintext pairs. The labels for these differences are binary, with values of 1 or 0. The generation of datasets can be found in Algorithm 1.

---

#### Algorithm 1: Data generation for training and validation

---

**Input:** Cipher  $E(\cdot)$ , Data Size  $N$ , Difference  $\delta$

**Output:** Training or Validation Dataset  $D$

```

1  $D \leftarrow \emptyset$ ;
2  $K \leftarrow \text{Random}$ ;
3 for  $i = 1$  to  $N$  do
4    $P_0, P_1 \leftarrow \text{Random}$ ;
5    $P_2 = P_1 \oplus \delta$ ;
6    $C_0 = E(P_0, K)$ ;
7    $C_1 = E(P_1, K)$ ;
8    $C_2 = E(P_2, K)$ ;
9   Append  $D$  with:
10   $(C_0 \oplus C_1, 0)$ ; // class 0
11   $(C_1 \oplus C_2, 1)$ ; // class 1
12 Return  $D$ 

```

---

##### 3.1.2. Network architecture and training pipeline

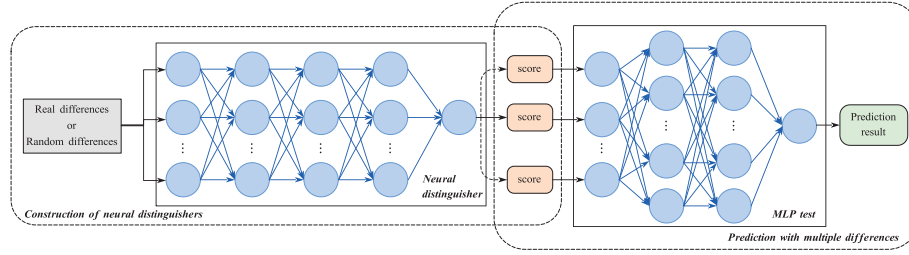
**Network Architecture:** The MLP network takes ciphertext differences as input, with 128-bit for GIFT-128 and 320-bit for ASCON-PERMUTATION. There are three hidden layers, with the same number of neurons as the input layer and ReLU activation function in each layer. The final layer outputs a real-valued score after a sigmoid activation function.

**Training Pipeline:** We take Binary Cross-Entropy Loss as the loss function since distinguishing differences can be seen as a binary classification task. Adam with default parameters in Keras is used as the optimizer. The training and validation of our neural distinguishers use the labeled datasets as described above. For each input ciphertext difference, the score between 0 and 1 calculated by the neural distinguisher representing the probability that the ciphertext difference is real. The distinguisher is built as follows:

- Step 1. Train the neural network with the training dataset. Classify the sample as 1 if the output score exceeds 0.5; otherwise, classify it as 0. Then calculate the training accuracy.
- Step 2. Start validation on the validation dataset if the training accuracy is higher than 50%. Or retrain a new neural network.

**Table 2**  
Bit permutation of GIFT-128.

| Index | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| $S_0$ | 29 | 25 | 21 | 17 | 13 | 9  | 5  | 1  | 30 | 26 | 22 | 18 | 14 | 10 | 6  | 2  |
| $S_1$ | 30 | 26 | 22 | 18 | 14 | 10 | 6  | 2  | 31 | 27 | 23 | 19 | 15 | 11 | 7  | 3  |
| $S_2$ | 31 | 27 | 23 | 19 | 15 | 11 | 7  | 3  | 28 | 24 | 20 | 16 | 12 | 8  | 4  | 0  |
| $S_3$ | 28 | 24 | 20 | 16 | 12 | 8  | 4  | 0  | 29 | 25 | 21 | 17 | 13 | 9  | 5  | 1  |
| Index | 15 | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| $S_0$ | 31 | 27 | 23 | 19 | 15 | 11 | 7  | 3  | 28 | 24 | 20 | 16 | 12 | 8  | 4  | 0  |
| $S_1$ | 28 | 24 | 20 | 16 | 12 | 8  | 4  | 0  | 29 | 25 | 21 | 17 | 13 | 9  | 5  | 1  |
| $S_2$ | 29 | 25 | 21 | 17 | 13 | 9  | 5  | 1  | 30 | 26 | 22 | 18 | 14 | 10 | 6  | 2  |
| $S_3$ | 30 | 26 | 22 | 18 | 14 | 10 | 6  | 2  | 31 | 27 | 23 | 19 | 15 | 11 | 7  | 13 |



**Fig. 1.** The overall pipeline for the proposed model of neural distinguishers.

**Table 3**

The round constants  $c_r$  used in each round of  $p^{12}$ .

| $p^{12}$ | Constant $c_r$ | $p^{12}$ | Constant $c_r$ |
|----------|----------------|----------|----------------|
| 0        | 0xf0           | 6        | 0x96           |
| 1        | 0xe1           | 7        | 0x87           |
| 2        | 0xd2           | 8        | 0x78           |
| 3        | 0xc3           | 9        | 0x69           |
| 4        | 0xb4           | 10       | 0x5a           |
| 5        | 0xa5           | 11       | 0x4b           |

Step 3. If the validation accuracy is lower than or equal to 50%, the training has been overfitting. Return to Step 1 or refine the data generation process if necessary.

Step 4. The neural network now is a valid neural distinguisher. Return the network with the lowest validation loss.

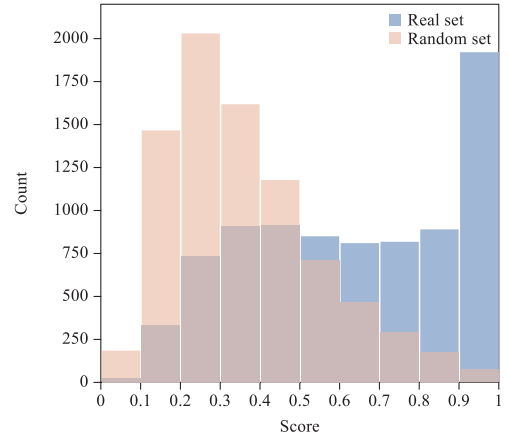
When testing the neural distinguisher with different sets, if the ciphertext differences in the set have the chosen input difference, the probability of classification as 1 (true positive rate) is expected to be lower than the validation accuracy. However, if the input differences of ciphertext differences are chosen randomly, the probability of classification as 0 (true negative rate) is likely to be higher than the validation accuracy. This is because the neural distinguisher can learn some significant features from the training data, but not all ciphertext differences have these features. Therefore, the neural distinguisher would classify ciphertext differences that do not conform to these features as random. In a word, distinguishing a specific input difference is generally more difficult than distinguishing random input differences.

### 3.2. Prediction with multiple ciphertext differences

#### 3.2.1. Motivations

When employing a neural distinguisher with validation accuracy slightly exceeding 50% for prediction, the classification accuracy for a set of real ciphertext differences will be equal to (or slightly lower than) 50%. It is hard to get convincing prediction results. However, we believe that this neural distinguisher has learned enough information to distinguish real differences from random differences.

We count the number of scores for ciphertext differences in each interval:  $[0, 0.1]$ ,  $[0.1, 0.2]$ , ...,  $[0.9, 1]$  for real and random sets. An example is shown in Fig. 2. We chose the neural distinguisher of



**Fig. 2.** Score distribution of real set and random set.

6-round GIFT-128 with 77.06% validation accuracy to show the distribution of scores more clearly in the histogram. In fact, for all neural distinguishers with a validation accuracy higher than 50%, there exists a significant distinction in the score distribution of real and random sets. Classification can be done by identifying the distribution of scores. Neural networks learn features of ciphertext differences in the training data, and output highly similar scores for differences with identical features. In addition, the distribution of differences in different sets of the same size exhibits remarkable similarity. Therefore, the object of classification should be a set of differences rather than a single difference.

#### 3.2.2. Classification based on the score distribution

So we decide to classify multiple ciphertext differences at once based on the score distribution, and the accuracy in the training and validation is only seen as an indicator of whether the training is effective. On a theoretical level, a neural distinguisher with a validation accuracy higher than 50% can be improved to achieve higher prediction accuracy.

It can be implemented with statistical tests, such as the Kolmogorov–Smirnov (K–S) test, which quantifies the distance between the

**Table 4**  
The 5-bit S-box of ASCON-PERMUTATION.

| x    | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| S(x) | 4  | 11 | 31 | 20 | 26 | 21 | 9  | 2  | 27 | 5  | 8  | 18 | 29 | 3  | 6  | 28 |
| x    | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| S(x) | 30 | 19 | 7  | 14 | 0  | 13 | 17 | 24 | 16 | 12 | 1  | 25 | 22 | 10 | 15 | 23 |

empirical distribution functions of two samples to check whether the two data samples come from the same distribution. A test set with multiple ciphertext differences is classified as follows: first, we generate a set of real differences and evaluate the scores of these differences with the neural distinguisher. This set of scores is seen as the stand score distribution of real differences. Then we compute the K-S test statistic and its  $p$ -value for scores of the test set and the stand distribution. If the  $p$ -value exceeds 0.05, the test set will be classified as real and random otherwise.

We need a large set of differences to improve the accuracy of neural distinguishers with the K-S test. However, it would increase the data complexity of prediction. In terms of this issue, we use MLP to learn the distribution of scores and then classify a test set with the information learned by the neural network. We refer to the method of checking the score distribution with MLP as the MLP test. This helps us to perform classification more efficiently.

### 3.2.3. MLP test

**Network Architecture:** Let  $s$  denote the number of ciphertext differences in a sample. The input layer consists of  $s$  units and receives scores evaluated by the neural distinguisher. There are two hidden layers, the number of neurons in each layer is 64 if  $s \leq 64$ , and  $s$  if  $s > 64$ . The activation function in hidden layers is ReLU. The final layer outputs a real-valued score after a sigmoid activation function.

**Training Pipeline:** The loss function is Binary Cross-Entropy Loss and the optimizer is Adam with default parameters in Keras. We train the MLP test by following steps:

1. Generate ciphertext differences with the chosen input difference and obtain their scores. Divide these scores into groups, each containing  $s$  scores, and then sort the  $s$  scores in ascending order (or descending order). Sorting aims to remove the positional relations of scores. Similarly, generate groups of scores for random ciphertext differences.
2. Generate training and validation dataset: half the data comes from score groups of real differences with label 1, and the other half is from score groups of random differences with label 0.
3. Train the neural network with training and validation dataset obtained as described above. If the validation accuracy is greater than that of the neural distinguisher, the MLP test is valid. If not, generate datasets with larger  $s$  and retrain.

We apply the MLP test and K-S test to our neural distinguishers for round-reduced GIFT-128 and ASCON-PERMUTATION in Sections 4.1 and 4.2. The results demonstrate the improvement of prediction with multiple differences to our neural distinguishers, and the MLP test is more efficient than the K-S test.

### 3.3. Comparison with previous neural distinguishers

**Gohr's Distinguishers.** Gohr [21] proposed distinguishers for SPECK32/64 [34] constructed with a deep residual network. In a further study of Gohr's work, Benamira [22] pointed out that Gohr's neural distinguishers effectively approximated the Difference Distribution Table (DDT) by the training of neural networks, and classified ciphertext pairs with this information.

For the training of neural distinguishers, Gohr generated training and validation data with a single input difference of the highest-probability transition found by their model. Differently, our neural

**Table 5**  
Differential characteristic of 12-round GIFT-128.

| Round       | 1  | 2  | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11  | 12       |
|-------------|----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|----------|
| $\log_2(p)$ | -5 | -7 | -10 | -12 | -17 | -25 | -37 | -41 | -47 | -52 | -57 | -62.4125 |

distinguishers only use the difference between the ciphertext pairs  $C \oplus C'$  while the input of Gohr's neural distinguishers takes a pair of ciphertext  $(C, C')$ . The experiments with group size  $s = 1$  in Section 4.4 serve as a comparison with Gohr's distinguishers model.

**Baksi et al.'s Distinguishers.** Baksi et al. [24] chose GIMLI [35], ASCON [32], KNOT [36], and CHASKEY [37] with bigger state for differential cryptanalysis, all of which are non-Markov ciphers. They designed two models to construct neural distinguishers with different networks.

The first model uses multiple input differences, selecting  $t$  ( $t \geq 2$ ) input differences to generate the dataset, regardless of random differences. It uses the differences between the ciphertext pairs  $C \oplus C'$  for training. The second model is similar to Gohr's distinguishers. We tried using the first model to train a neural distinguisher for GIFT-128, but the results were inferior to those achieved with our single input difference neural distinguishers. As we discussed at the end of Section 3.1, the features of ciphertext differences with a fixed input difference that the neural distinguishers can learn are limited. The validation accuracy of a neural distinguisher with two given input differences will be lower than that with one given difference and random differences. Especially when the differential features from two given input differences have overlapping parts.

**Chen et al.'s Distinguishers.** Chen et al. [27] proposed a new distinguishers model considering features derived from multiple ciphertext pairs. Based on this model they improved the accuracy of distinguishers for SPECK32/64.

This new model can be seen as an improvement of Gohr's distinguishers, which take multiple ciphertext pairs as the input of the network. Their new distinguishers captured features derived from multiple ciphertext pairs, and similarly, our proposed model improves the accuracy of distinguishers with multiple ciphertext differences. In Section 4.4, we conduct some experiments to compare the results of Chen et al.'s model on GIFT-128 and ASCON-PERMUTATION with ours.

## 4. Experiments and analysis<sup>1</sup>

### 4.1. Neural distinguishers for round-reduced GIFT-128

Using an improved MILP-based approach, Zhu et al. [38] presented some differential characteristics for reduced round GIFT-128. A 12-round differential characteristic is shown in Table 5, and the data complexity of the 7-round classical differential distinguisher is  $2^{37}$ . We construct 7-round neural distinguishers with datasets of size  $2^{23.2}(10^7)$  in the training and validation based on the proposed model.

Now, we present our neural distinguishers for 5 to 7 rounds of GIFT-128.

**Training:** We assume that there is only one plaintext block for the data collection. Based on our data generation algorithm, we generate uniformly distributed 128-bit keys to compute 128-bit ciphertext differences with a difference of 1 in the 7th byte of plaintext pairs,

<sup>1</sup> The code is available at <https://github.com/yijSong/ND-GIFT-ASCON>.

**Table 6**

Accuracy of neural distinguishers for round-reduced GIFT-128.

| Round | Validation accuracy |
|-------|---------------------|
| 5     | 96.43%              |
| 6     | 77.06%              |
| 7     | 55.42%              |

**Table 7**

Prediction accuracy of improved neural distinguishers for round-reduced GIFT-128.

| Round | Method   | $s$ | Prediction accuracy |
|-------|----------|-----|---------------------|
| 6     | K-S test | 4   | 83.01%              |
|       |          | 8   | 95.55%              |
|       |          | 16  | 98.14%              |
|       | MLP test | 2   | 85.84%              |
|       |          | 4   | 93.87%              |
|       |          | 8   | 98.59%              |
| 7     | K-S test | 64  | 56.92%              |
|       |          | 128 | 65.13%              |
|       |          | 256 | 86.59%              |
|       |          | 512 | 98.31%              |
|       | MLP test | 4   | 61.73%              |
|       |          | 8   | 66.62%              |
|       |          | 16  | 73.24%              |
|       |          | 32  | 81.06%              |
|       |          | 64  | 87.26%              |
|       |          | 128 | 91.56%              |
|       |          | 256 | 95.88%              |
|       |          | 512 | 99.36%              |

along with 128-bit random differences. Training is run for 50 epochs. The results on the validation data are given in Table 6. Note that the validation accuracy only indicates the results of training, not the prediction accuracy.

**Prediction:** We use the proposed method to improve the prediction accuracy of neural distinguishers for 6 to 7 rounds of GIFT-128 because the accuracy of the neural distinguisher for 5 rounds is high enough. Both the MLP test and K-S test are used here respectively.

We denote  $s$  as the group size of a differences sample. For the K-S test, we first generate  $s$  (for  $s \leq 128$  the number is 128) real ciphertext differences and evaluate their scores by our distinguishers. Then we randomly generate 50 000 real samples and 50 000 random samples for the evaluation.

For the MLP test, we generate  $10^7/s$  real and random samples for the training and  $10^6/s$  real and random samples for the validation. Table 7 shows the results of the experiment.

**Analysis:** We achieve high prediction accuracy with small samples using the MLP test for 6 rounds of GIFT-128. For the K-S test, we need a larger  $s$  to obtain an accuracy close to the MLP test. MLP test can detect the score distribution in smaller samples because it learns features of both real and random differences scores for classification. While the K-S test just uses scores of real differences as the stand score distribution to evaluate samples. Therefore MLP test requires a larger dataset for training. For the K-S test, a large real sample is not necessary, it is better to take a real sample that is close in size to  $s$ .

For the neural distinguisher of 7 rounds GIFT-128 with a low validation accuracy, both the MLP test and K-S test require larger samples to improve the prediction accuracy. MLP test achieves better overall accuracy than the K-S test with the same size of  $s$ .

#### 4.2. Neural distinguishers for round-reduced ASCON

Using heuristic search tools, the authors of ASCON [39] presented an optimal differential transition with the differential probability of  $2^{-107}$  for 4-round ASCON-PERMUTATION. Baksı et al. [24] presented machine learning-based differential distinguishers with a data complexity of  $2^{19}$  for up to 3 rounds of ASCON-PERMUTATION based on their first model (two input differences), which can be found in Table 8(a).

**Table 8**

Accuracy of neural distinguishers for round-reduced ASCON-PERMUTATION.

| (a) Baksı's |                     | (b) Ours |                     |
|-------------|---------------------|----------|---------------------|
| Round       | Validation accuracy | Round    | Validation accuracy |
| 1           | 87.49%              | 1        | 100%                |
| 2           | 99.90%              | 2        | 100%                |
| 3           | 98.61%              | 3        | 99.99%              |
| 4           | –                   | 4        | 50.69%              |

**Table 9**

Prediction accuracy of the improved neural distinguisher for 4-round of ASCON-PERMUTATION.

| Round | Method   | $s$   | Prediction accuracy |
|-------|----------|-------|---------------------|
| 4     | K-S test | 1024  | 50.25%              |
|       |          | 2048  | 51.04%              |
|       |          | 4096  | 64.80%              |
|       |          | 8192  | 67.58%              |
|       |          | 16384 | 81.25%              |
|       |          | 32768 | 97.52%              |
|       | MLP test | 65536 | 99.47%              |
|       |          | 64    | 53.76%              |
|       |          | 128   | 55.21%              |
|       |          | 256   | 58.02%              |
|       |          | 512   | 61.61%              |
|       |          | 1024  | 65.61%              |
|       |          | 2048  | 69.25%              |

Applying our new model, we obtain neural distinguishers for up to 4 rounds of ASCON-PERMUTATION with a single input difference. The size of training and validation data is  $2^{19.9}(10^6)$  for 1 to 3 rounds, and  $2^{23.2}(10^7)$  for 4-round.

**Training:** Because the ASCON-PERMUTATION does not require a key for computation, we only need to generate uniformly distributed 320-bit states. The difference (0000 0001) is XOR to the last byte of the 64-bit register  $x_0$  as the chosen input difference. Then we input 320-bit differences to the MLP for training. Training is conducted for 20 epochs. Table 8(b) shows our results.

It can be found that our neural distinguishers achieve higher validation accuracy than Baksı's for 1 to 3 rounds of ASCON-PERMUTATION and cover one additional round.

**Prediction:** In order to know whether our new method improves the prediction accuracy of the neural distinguisher for 4 rounds of ASCON-PERMUTATION with a validation accuracy slightly higher than 50%, Table 9 shows an experiment similar to that in Section 4.1. In addition to sorting the scores within the samples, we also perform Min–Max Normalization on the scores in each sample for the MLP test. We observe that the scores within the samples are densely distributed between 0.47 and 0.52. This normalization aids in better learning of the characteristics of the score distribution and ensures sufficient gradient for convergence.

**Analysis:** We need to generate differences samples with a minimum size of  $2^8$  to effectively improve the prediction accuracy using the MLP test, and for the K-S test the minimum size is  $2^{13}$ . The training for the MLP test with more than 2048 input units is hard to fit, so the highest accuracy is 69.25% with test sets of size 2048. But we can still achieve higher accuracy with the K-S test, despite the large size of  $s$ . The signal from this neural distinguisher is rather weak as the scores are densely distributed around 0.5. But the distinguisher has indeed learned certain features proved by the results of the K-S test.

#### 4.3. Discussion of the input differences

It can be determined that the accuracy of distinguishers is significantly influenced by the choice of input differences. (0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0706, 0x0, 0x0) is the input difference of a high probability 14-round differential characteristic of GIFT-128 found in [38],

**Table 10**

Experiment with Different Input Differences for 6-round GIFT-128.

| Hamming weight | [50%, 55%) | [55%, 60%) | [60%, 65%) | [65%, 70%) | [70%, 75%) | [75%, 80%) |
|----------------|------------|------------|------------|------------|------------|------------|
| 1              | 0          | 0          | 0          | 32         | 72         | 24         |
| 2              | 67         | 48         | 5          | 4          | 2          | 2          |
| 3              | 122        | 5          | 1          | 0          | 0          | 0          |
| 4              | 127        | 1          | 0          | 0          | 0          | 0          |
| 5              | 128        | 0          | 0          | 0          | 0          | 0          |

**Table 11**

Comparison of distinguishers trained with different models.

| (a) 6 and 7 rounds GIFT-128   |                      |         |         |         |         |          |          |
|-------------------------------|----------------------|---------|---------|---------|---------|----------|----------|
| Round                         | Distinguisher        | $s = 1$ | $s = 2$ | $s = 4$ | $s = 8$ | $s = 16$ | $s = 32$ |
| 6                             | $\mathcal{N}D_{Res}$ | 78.36%  | 84.77%  | 88.00%  | 87.12%  |          |          |
|                               | $\mathcal{N}D_{MLP}$ | 77.06%  | 85.84%  | 93.87%  | 98.59%  |          |          |
| 7                             | $\mathcal{N}D_{Res}$ | 56.06%  | 55.64%  | 50.17%  | 50.06%  |          |          |
|                               | $\mathcal{N}D_{MLP}$ | 55.42%  | 58.04%  | 61.73%  | 66.62%  |          |          |
| (b) 4-round ASCON-PERMUTATION |                      |         |         |         |         |          |          |
| Round                         | Distinguisher        | $s = 1$ | $s = 2$ | $s = 4$ | $s = 8$ | $s = 16$ | $s = 32$ |
| 4                             | $\mathcal{N}D_{Res}$ | 50.56%  | 50.54%  | 50.18%  | 50.09%  | 50.27%   | 50.02%   |
|                               | $\mathcal{N}D_{MLP}$ | 50.69%  | 50.91%  | 51.21%  | 51.56%  | 52.12%   | 52.63%   |

from which we only obtain a 6-round distinguisher with an accuracy of 50.77%. And our most effective distinguishers for GIFT-128 and ASCON-PERMUTATION are obtained by using input differences with hamming weight 1. Therefore, we design an experiment to compare the effect of input differences with different hamming weights on the accuracy of distinguishers. The task of this experiment is training distinguishers for 6-round GIFT-128, as we can observe the distribution of accuracy over a larger interval. We use the input differences with hamming weight of 1 to 5 for the training. We first generate all the input differences with hamming weight 1, totaling 128 differences. Then, for the experiments of input differences with hamming weight of 2 to 5, we randomly generate 128 differences for each without repeating. After training distinguishers with these input differences, we count the number of distinguishers' accuracies distributed in each interval. The results are shown in Table 10.<sup>2</sup>

It can be seen that as the hamming weight increases, more distinguishers achieve low accuracy. Hou et al. [40] proposed that the higher the Hamming weight of the input difference, the weaker the non-random feature of the ciphertext pair. This is partially confirmed by our experimental results. Therefore when choosing input difference for the training of distinguishers, starting with an input difference with hamming weight 1 is more likely to achieve high accuracy.

#### 4.4. Comparison with Chen et al.'s distinguisher model

To compare with the distinguisher model proposed by Chen et al. [27] with Residual Network, we train the distinguishers for 6 to 7 rounds of GIFT-128 and 4 rounds of ASCON-PERMUTATION with their model. The code of network construction and training is from [27]. For convenience, we denote the distinguishers trained with Chen et al.'s model as  $\mathcal{N}D_{Res}$  and ours as  $\mathcal{N}D_{MLP}$ .

The size of a training set is  $N = 10^7$ , and the size of a validation set is  $M = 10^6$ . Each sample consists of  $s$  ciphertext differences for  $\mathcal{N}D_{MLP}$  and  $s$  ciphertext pairs for  $\mathcal{N}D_{Res}$ . A training set is composed of  $N/2s$  real samples and  $N/2s$  random samples, and a validation set is composed of  $M/2s$  real samples and  $M/2s$  random samples. Training is conducted for 50 epochs, and the best validation accuracy is recorded. The results are shown in the Table 11.

For 6-round GIFT-128: When  $s = 1$ ,  $\mathcal{N}D_{Res}$  achieves a higher accuracy than  $\mathcal{N}D_{MLP}$  due to the use of ciphertext pairs as input. As

$s$  increases to 2 and 4,  $\mathcal{N}D_{Res}$  has an improved accuracy and  $\mathcal{N}D_{MLP}$  brings a higher accuracy improvement. When  $s = 8$ , the accuracy of  $\mathcal{N}D_{Res}$  decreases because of the overfitting, whereas  $\mathcal{N}D_{MLP}$  still can improve its accuracy.

For 7-round GIFT-128:  $\mathcal{N}D_{Res}$  achieves a similar accuracy to  $\mathcal{N}D_{MLP}$  with  $s = 1$ . When  $s = 2$ ,  $\mathcal{N}D_{Res}$  is unable to improve the accuracy, and its accuracy decreases with  $s \geq 4$  due to overfitting. Meanwhile,  $\mathcal{N}D_{MLP}$  keeps improving the accuracy as  $s$  increases.

For 4-round ASCON-PERMUTATION:  $\mathcal{N}D_{MLP}$  with  $s = 1$  performs similarly to  $\mathcal{N}D_{Res}$ ,  $s = 1, 2$ . When  $s \geq 4$ ,  $\mathcal{N}D_{Res}$  cannot achieve a valid result because of the overfitting. We observe that with the increase in cipher state size,  $\mathcal{N}D_{Res}$  overfits more severely when  $s \geq 2$ . This is also the reason why  $\mathcal{N}D_{MLP}$ , which takes ciphertext differences as input, obtains higher accuracy than  $\mathcal{N}D_{Res}$  on GIFT-128 and ASCON-PERMUTATION when  $s \geq 2$ .

It is worth noticing that we do not perform Min-Max Normalization on the training and validation sets for our  $\mathcal{N}D_{MLP}$  of 4-round ASCON-PERMUTATION at  $s = 2$  to  $s = 32$ , and instead achieve higher accuracy compared to the training with normalized sets. We initially suspect this might be an incidental occurrence on the current validation set, as the validation accuracy fluctuates between the highest accuracy and 0.5 during training. Therefore, we save the model with the highest accuracy for evaluation, and the obtained accuracy is very close to the previously observed one. What has the  $\mathcal{N}D_{MLP}$  learned from these few scores closely distributed around 0.5? It is worth further investigation in future research.

## 5. Conclusion

In this paper, we proposed a new model for the construction of neural differential distinguishers for lightweight ciphers, including a new method called the MLP test for the classification of ciphertext differences. This work improves the performance of the previous neural distinguishers for 4-round ASCON-PERMUTATION and constructs the neural distinguishers of 7-round GIFT-128 for the first time. The object of classification is changed from a single ciphertext difference to multiple ciphertext differences. We trained neural networks to classify the distribution of scores, and the K-S test is also used for prediction to show the comparison with the MLP test. Predicting based on the score distribution of ciphertext differences greatly improves the accuracy of prediction, with our new method the MLP test proving more efficient than the K-S test. By comparing with the existing distinguishers model using multiple ciphertext pairs, our new model is found to be more suitable for large state ciphers such as GIFT-128 and ASCON-PERMUTATION.

<sup>2</sup> The input differences we generate and the corresponding accuracy are given in our GitHub repository.

Neural distinguishers do not rely on the optimal differential characteristic and they can reduce the data complexity by learning the distribution of ciphertext differences. Compared with the input difference of a high-probability differential transition, an input difference with hamming weight 1 is more likely to help neural distinguishers achieve higher accuracy.

As future work, we will continue optimizing the proposed model to cover further rounds and improve the distinguishing accuracy. Accordingly, we can give a more powerful illustration of the feasibility of applying neural networks to cryptanalysis.

#### CRedit authorship contribution statement

**Dongsu Shen:** Conceptualization, Writing – review & editing. **Yijian Song:** Methodology, Software, Writing – original draft. **Yuan Lu:** Validation, Visualization. **Saiqin Long:** Formal analysis. **Shujuan Tian:** Data curation, Project administration.

#### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

#### Data availability

Data will be made available on request.

#### Acknowledgments

This work was supported in part by the National Key R&D Program of China (Grant No. 2022YFB2701600); the Xiangtan University scientific research project (Grant No. 15XZX32); the National Natural Science Foundation of China (Grant No. 62172350); National Natural Science Foundation of China (Grant No. 62172349); National Natural Science Foundation of Hunan Province (Grant No. 2023JJ30597); the Research Foundation of Education Bureau of Hunan Province (Grant No. 21B0139); Open Project of the State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences (Grant No. SYSKF2101).

#### References

- [1] Dinu Daniel, Corre Yann Le, Khovratovich Dmitry, Perrin Léo, Großschädl Johann, Biryukov Alex. Triathlon of lightweight block ciphers for the internet of things. *J Cryptogr Eng* 2019;9:283–302.
- [2] Bhasin Akshay, Mishra Girish. Recent advances in lightweight stream ciphers. *CSI Trans ICT* 2016;4:173–6.
- [3] Katagi Masanobu, Moriai Shihō, et al. Lightweight cryptography for the internet of things. *Sony Corp* 2008;2008:7–10.
- [4] Megha Mukundan Puliparambil, Manayankath Sindhu, Srinivasan Chungath, Sethumadhavan Madathil. Hash-one: a lightweight cryptographic hash function. *IET Inf Secur* 2016;10(5):225–31.
- [5] Sankaran Sriram. Lightweight security framework for IoTs using identity based cryptography. In: 2016 international conference on advances in computing, communications and informatics. ICACCI, IEEE; 2016, p. 880–6.
- [6] Singh Saurabh, Sharma Pradip Kumar, Moon Seo Yeon, Park Jong Hyuk. Advanced lightweight encryption algorithms for IoT devices: survey, challenges and solutions. *J Ambient Intell Humaniz Comput* 2017;1–18.
- [7] Daemen Joan, Rijmen Vincent. The design of Rijndael. Vol. 2, Springer; 2002.
- [8] Philip Merly Annie, et al. A survey on lightweight ciphers for IoT devices. In: 2017 international conference on technological advancements in power and energy (TAP energy). IEEE; 2017, p. 1–4.
- [9] McKay Kerry, Bassham Lawrence, Sönmez Turan Meltem, Mouha Nicky. Report on lightweight cryptography. Technical report, National Institute of Standards and Technology; 2016.
- [10] Turan Meltem Sönmez, McKay Kerry A, Çalik Çağdas, Chang Donghoon, Bassham Lawrence, et al. Status report on the first round of the NIST lightweight cryptography standardization process. Vol. 108, National Institute of Standards and Technology, Gaithersburg, MD, NIST Interagency/Internal Rep.(NISTIR); 2019.
- [11] Turan Meltem Sönmez, McKay Kerry, Chang Donghoon, Çalik Çağdas, Bassham Lawrence, Kang Jinkeon, Kelsey John, et al. Status report on the second round of the NIST lightweight cryptography standardization process. Vol. 8369, National Institute of Standards and Technology Internal Report; 2021, (10.6028).
- [12] Biham Eli, Shamir Adi. Differential cryptanalysis of DES-like cryptosystems. *J Cryptol* 1991;4:3–72.
- [13] Matsui Mitsuru. Linear cryptanalysis method for DES cipher. In: Advances in cryptography—EUROCRYPT'93: workshop on the theory and application of cryptographic techniques Iofthus, Norway, May 23–27, 1993 proceedings 12. Springer; 1994, p. 386–97.
- [14] Mouha Nicky, Wang Qingju, Gu Dawu, Preneel Bart. Differential and linear cryptanalysis using mixed-integer linear programming. In: Information security and cryptology: 7th international conference, inscrypt 2011, Beijing, China, November 30–December 3, 2011. revised selected papers 7. Springer; 2012, p. 57–76.
- [15] Isola Phillip, Zhu Jun-Yan, Zhou Tinghui, Efros Alexei A. Image-to-image translation with conditional adversarial networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2017, p. 1125–34.
- [16] Strubell Emma, Ganesh Ananya, McCallum Andrew. Energy and policy considerations for deep learning in NLP. 2019, arXiv preprint arXiv:1906.02243.
- [17] Chen Xiaozhi, Ma Huimin, Wan Ji, Li Bo, Xia Tian. Multi-view 3d object detection network for autonomous driving. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2017, p. 1907–15.
- [18] Rivest Ronald L, et al. Cryptography and machine learning. In: ASIACRYPT. 1991, p. 427–39.
- [19] Maghrebi Houssein, Portigliatti Thibault, Prouff Emmanuel. Breaking cryptographic implementations using deep learning techniques. In: Security, privacy, and applied cryptography engineering: 6th international conference, SPACE 2016, Hyderabad, India, December 14–18, 2016, proceedings 6. Springer; 2016, p. 3–26.
- [20] Cagli Eleonora, Dumas Cécile, Prouff Emmanuel. Convolutional neural networks with data augmentation against jitter-based countermeasures: Profiling attacks without pre-processing. In: Cryptographic hardware and embedded systems—CHES 2017: 19th international conference, Taipei, Taiwan, September 25–28, 2017, proceedings. Springer; 2017, p. 45–68.
- [21] Gohr Aron. Improving attacks on round-reduced speck32/64 using deep learning. In: Advances in cryptography—CRYPTO 2019: 39th annual international cryptography conference, Santa Barbara, CA, USA, August 18–22, 2019, proceedings, part II 39. Springer; 2019, p. 150–79.
- [22] Benamira Adrien, Geraud David, Peyrin Thomas, Tan Quan Quan. A deeper look at machine learning-based cryptanalysis. In: Advances in cryptography—EUROCRYPT 2021: 40th annual international conference on the theory and applications of cryptographic techniques, Zagreb, Croatia, October 17–21, 2021, proceedings, part I 40. Springer; 2021, p. 805–35.
- [23] Su Heng-Chuan, Zhu Xuan-Yong, Ming Duan. Polytropic attack on round-reduced simon32/64 using deep learning. In: Information security and cryptology: 16th international conference, inscrypt 2020, Guangzhou, China, December 11–14, 2020, revised selected papers. Springer; 2021, p. 3–20.
- [24] Baksı Anubhab, Baksı Anubhab. Machine learning-assisted differential distinguishers for lightweight ciphers. *Class Phys Secur Symmetric Key Cryptogr Algorithms* 2022;141–62.
- [25] Rajan Reshma, Roy Rupam Kumar, Sen Diptakshi, Mishra Girish. Deep learning-based differential distinguisher for lightweight cipher GIFT-cofb. In: Machine intelligence and smart systems: proceedings of MISS 2021. Springer; 2022, p. 397–406.
- [26] Wang Meiqin. Differential cryptanalysis of reduced-round PRESENT. *Lecture Notes in Comput Sci* 2008;5023:40–9.
- [27] Chen Yi, Shen Yantian, Yu Hongbo, Yuan Sitong. A new neural distinguisher considering features derived from multiple ciphertext pairs. *Comput J* 2023;66(6):1419–33.
- [28] Bao Zhenzhen, Lu Jinyu, Yao Yiran, Zhang Liu. More insight on deep learning-aided cryptanalysis. In: International conference on the theory and application of cryptography and information security. Springer; 2023, p. 436–67.
- [29] Liu JiaShuo, Ren JiongJiong, Chen ShaoZhen, Li ManMan. Improved neural distinguishers with multi-round and multi-splicing construction. *J Inf Secur Appl* 2023;74:103461.
- [30] Teng Wei Jian, Teh Je Sen, Jamil Norziana. On the security of lightweight block ciphers against neural distinguishers: Observations on LBC-IoT and SLIM. *J Inf Secur Appl* 2023;76:103531.
- [31] Banik Subhadeep, Chakraborti Avik, Inoue Akiko, Iwata Tetsu, Mine-matsu Kazuhiko, Nandi Mrudul, Peyrin Thomas, Sasaki Yu, Sim Siang Meng, Todo Yosuke. Gift-cofb. *Cryptol ePrint Arch* 2020.
- [32] Dobraunig Christoph, Eichlseder Maria, Mendel Florian, Schläffer Martin. Ascon v1. 2: lightweight authenticated encryption and hashing. *Journal of Cryptology* 2021;34:1–42.
- [33] Bertoni Guido, Daemen Joan, Peeters Michaël, Van Assche Gilles. Duplexing the sponge: single-pass authenticated encryption and other applications. In: Selected areas in cryptography: 18th international workshop, SAC 2011, Toronto, ON, Canada, August 11–12, 2011, revised selected papers 18. Springer; 2012, p. 320–37.

- [34] Beaulieu Ray, Shors Douglas, Smith Jason, Treatman-Clark Stefan, Weeks Bryan, Wingers Louis. The SIMON and SPECK lightweight block ciphers. In: Proceedings of the 52nd annual design automation conference. 2015, p. 1–6.
- [35] Bernstein Daniel J, Kölbl Stefan, Lucks Stefan, Massolino Pedro Maat Costa, Mendel Florian, Nawaz Kashif, Schneider Tobias, Schwabe Peter, Standaert François-Xavier, Todo Yosuke, et al. Gimli: a cross-platform permutation. In: Cryptographic hardware and embedded systems—CHES 2017: 19th international conference, taipei, Taiwan, September 25-28, 2017, proceedings. Springer; 2017, p. 299–320.
- [36] Zhang Wentao, Ding Tianyou, Yang Bohan, Bao Zhenzhen, Xiang Zejun, Ji Fulei, Zhao Xuefeng. KNOT: algorithm specifications and supporting document. Submission to NIST lightweight cryptography project 2019.
- [37] Mouha Nicky. Chaskey: a MAC algorithm for microcontrollers—status update and proposal of chaskey-12–. Cryptol ePrint Arch 2015.
- [38] Zhu Baoyu, Dong Xiaoyang, Yu Hongbo. MILP-based differential attack on round-reduced GIFT. In: Topics in cryptology—CT-RSA 2019: the cryptographers' track at the RSA conference 2019, san francisco, CA, USA, March 4–8, 2019, proceedings. Springer; 2019, p. 372–90.
- [39] Dobraunig Christoph, Eichlseder Maria, Mendel Florian, Schläffer Martin. Cryptanalysis of ascon. In: Topics in cryptology—CT-RSA 2015: the cryptographer's track at the RSA conference 2015, san francisco, CA, USA, April 20-24, 2015. proceedings. Springer; 2015, p. 371–87.
- [40] Hou Zezhou, Ren Jiongjiong, Chen Shaozhen. Improve neural distinguisher for cryptanalysis. Cryptol ePrint Arch 2021.